



UTNMDP
Regional Mar del Plata

Trabajo Práctico Grupal
Base de Datos II

Bases de datos NoSQL

Integrantes:

- Lomuoio, Lucas
- Rossitto, Sofía
- Scalzo, Stephano



2026

Índice

1. Introducción	3
2. Bases de datos NoSQL	4
2.1. Comparación entre SQL y NoSQL	4
2.2. Tipos de bases de datos NoSQL	5
3. Bases de Datos de Documentos	6
3.1. Características	6
3.2. Ventajas y desventajas	7
4. MongoDB	7
4.1. Orígenes	8
4.2. Modelo de Datos	8
4.3. Formato JSON frente a BSON	9
4.4. Operaciones básicas	9
4.5. Ventajas	10
4.6. Desventajas	10
4.7. Casos de uso	11
4.8. Ejemplo práctico	11
5. Comparación con bases de datos SQL	13
5.1. Criterios en las Querys	13
5.2. Evolución del esquema	14
5.2.1. Escalabilidad	15
5.2.2. Consistencia	16
5.2.3. Transacciones	16
5.3. Comparación de rendimiento entre NoSQL y SQL	17
6. Conclusión	18
7. Tabla final de repaso	19
8. Bibliografía	20

1. Introducción

Una base de datos es una recopilación organizada de información, que se suele almacenar de forma electrónica en un sistema informático. Normalmente, una base de datos está controlada por un sistema de gestión de bases de datos (DBMS). En conjunto, los datos y el DBMS, junto con las aplicaciones asociadas a ellos, reciben el nombre de sistema de bases de datos. Dos de los tipos de bases de datos más empleados son las bases de datos SQL (Structured Query Language) y las bases de datos NoSQL (Not only SQL).

Bases de datos SQL

Se encuentran bajo un tipo de base de datos denominada bases de datos relacionales. Estas almacenan datos dentro de filas y columnas que se utilizan para formar tablas. Se puede crear una relación entre las dos tablas (o más) empleando una clave externa (claves foráneas) que mantienen relaciones predefinidas que existen entre las tablas.

Bases de datos NoSQL

NoSQL, que significa Not only SQL, surgieron como alternativa para almacenar datos no estructurados o semiestructurados que no se adaptan fácilmente al modelo relacional. Ofrecen esquemas flexibles y distintos modelos de organización de datos.

Objetivo del trabajo

El presente trabajo tiene como objetivo introducir el concepto de bases de datos NoSQL, analizar sus principales características y profundizar en el modelo de bases de datos documentales, tomando como caso de estudio a MongoDB.

2. Bases de datos NoSQL

Las bases de datos NoSQL surgieron como una alternativa a las bases de datos relacionales tradicionales, especialmente frente a la necesidad de almacenar grandes volúmenes de datos heterogéneos y de alta variabilidad.

SQL es un lenguaje específico de dominio que se utiliza para consultar y gestionar datos. Permite a los usuarios aplicar lógica compleja mediante transacciones y procedimientos integrados, como funciones almacenadas o vistas.

Los datos no estructurados, como las publicaciones en las redes sociales, no tienen un modelo de datos predefinido ni una organización coherente. Pueden actualizarse y cambiar rápidamente, mientras que los datos estructurados, como las transacciones bancarias, tienen una tasa de cambio menor. Algunos ejemplos de datos no estructurados son las imágenes, los audios, los videos y los mapas.

Escalabilidad horizontal

Una de las principales características de las bases de datos NoSQL es la escalabilidad horizontal, que permite distribuir los datos entre múltiples servidores. Esto posibilita manejar grandes volúmenes de información y altos niveles de tráfico de manera eficiente.

2.1. Comparación entre SQL y NoSQL

La principal ventaja de usar un sistema NoSQL es que permite a los desarrolladores almacenar y acceder a los datos de forma rápida y sencilla, sin la complejidad de una base de datos relacional tradicional. Sin embargo, la elección entre SQL y NoSQL no depende solo del rendimiento, sino del tipo de datos, el patrón de acceso, los índices y los requisitos de consistencia de cada sistema.

En general, las bases de datos SQL son adecuadas para datos estructurados, donde la información es consistente y las relaciones entre tablas están bien definidas. En cambio, las bases de datos NoSQL son adecuadas para datos semiestructurados o no estructurados, donde los datos no se ajustan a un esquema predefinido y las relaciones entre los elementos no están bien definidas.

Criterio	SQL	NoSQL
Estructura de datos	Tablas con filas y columnas de esquema fijo	Documentos, grafos, clave-valor, columnas anchas. Estructura flexible
Esquema	Rígido: todos los registros comparten las mismas columnas	Dinámico: cada documentos puede tener campos distintos
Escalabilidad	Principalmente vertical(más CPU/RAM en el mismo servidor)	Principalmente horizontal(más servidores); depende del motor y la configuración
Consultas	SQL estándar con JOINS complejos; ideal para relaciones entre entidades	APIs o lenguajes propios; eficiente para lectura de una sola entidad.
Consistencia	ACID fuerte: garantiza consistencia inmediata en cada operación	Puede ser eventual: prioriza disponibilidad y rendimiento sobre consistencia inmediata
Casos ideales	Transacciones financieras, datos altamente estructurados con relaciones definidas	Aplicaciones web/móviles, big data, catálogos con atributos variables, redes sociales

2.2. Tipos de bases de datos NoSQL

Las bases de datos NoSQL ofrecen distintos modelos para organizar y almacenar la información, adaptándose a diferentes necesidades y tipos de aplicaciones. Al ofrecer diversas estructuras de datos, NoSQL puede aplicarse al análisis de datos, la gestión de big data, las redes sociales y el desarrollo de aplicaciones móviles.

Los tipos comunes de bases de datos no relacionales incluyen:

Grafos	Documentos	Clave-valor	Columna ancha
<i>ej: Neo4J</i>	<i>ej: MongoDB</i>	<i>ej: Redis</i>	<i>ej: Cassandra</i>
Almacenan datos como nodos (entidades) y edges (relaciones). Ideales para rastrear relaciones	Almacenan datos como documentos JSON y BSON. comunes en los sistemas de gestión de contenidos	Almacenan la información como pares clave-valor. Claves únicas con matrices de datos como valores	Muy parecida a las bases de datos relacionales pero cada fila puede tener su propio conjunto distinto de columnas

3. Bases de Datos de Documentos

Una base de datos de documentos es un tipo NoSQL que almacena datos en "documentos" flexibles, que suelen tener formatos como JSON o BSON.

Analogía del archivador

Una forma de entender las bases de datos de documentos es pensándolas como si fuera un archivador. Los datos se agruparían en colecciones, que son las carpetas, y cada una de ellas contiene documentos independientes, que son los archivos, de una sola entidad. Los documentos almacenan datos en pares campo-valor. Los valores pueden ser de diversos tipos y estructuras, como cadenas de texto, números, fechas, matrices u objetos.

3.1. Características

Característica	Descripción
Modelo	Los datos se almacenan en documentos que se corresponden con objetos en la mayoría de los lenguajes de programación, lo que permite a los desarrolladores crear aplicaciones rápidamente.
Esquema	Tienen esquemas flexibles: no todos los documentos de una colección tienen que tener los mismos campos. Cabe destacar que algunas bases de datos de documentos admiten la validación de esquemas ,por lo que el esquema puede bloquearse opcionalmente.
Distribuidas	Las bases de datos de documentos son distribuidas, lo que permite la escalabilidad horizontal, generalmente es más económico que la vertical.
Resiliencia	Las bases de datos de documentos ofrecen resiliencia mediante la replicación lo que significa que tiene la capacidad de sobrevivir, adaptarse y recuperarse rápidamente ante fallos, garantizando que la información crítica siga disponible y sin corromperse.
Consultas	Las bases de datos de documentos cuentan con APIs o lenguajes de consulta que permite a los desarrolladores ejecutar operaciones CRUD (create, read, update y delete).

3.2. Ventajas y desventajas

Ventajas	Desventajas
Flexibilidad: esquemas dinámicos para modelos de datos cambiantes	Redundancia de datos: duplicidad por la flexibilidad de esquemas
Rendimiento: indexación integrada y patrones de acceso eficientes	Relaciones complejas: no diseñadas para relaciones entre grandes cantidades de datos
Escalabilidad: horizontal (más servidores) y vertical (más potencia computacional)	Consistencia: prioriza rendimiento sobre consistencia inmediata
Experiencia de desarrolladores: documentos JSON con sintaxis familiar	
Versatilidad: manejo de datos no estructurados, semiestructurados y metadatos	

4. MongoDB

MongoDB es una base de datos NoSQL de código abierto diseñada para trabajar con datos orientados a documentos. En lugar de filas y columnas tradicionales, almacena los datos en documentos flexibles similares a JSON, lo que lo hace ideal para aplicaciones que necesitan evolucionar rápidamente.

Los documentos o colecciones de documentos de MongoDB son las unidades básicas de datos. Con formato BSON (Binary Java Script Object Notation), estos documentos pueden almacenar varios tipos de datos y distribuirse en múltiples sistemas. Dado que MongoDB emplea un diseño de esquema dinámico, los usuarios tienen una flexibilidad sin precedentes a la hora de crear registros de datos, consultar colecciones de documentos a través de la agregación de MongoDB y analizar grandes cantidades de información.

4.1. Orígenes

Historia de MongoDB

Dwight Merriman, Eliot Horowitz y Kevin Ryan, veteranos de DoubleClick, comprendieron las limitaciones de las bases de datos relacionales al escalar la empresa para gestionar más de 400 000 anuncios por segundo. Basándose en su experiencia, creyeron poder crear una solución más escalable y flexible para almacenar y gestionar datos a gran volumen y velocidad, con el fin de aumentar la productividad de los desarrolladores y mejorar las operaciones comerciales. El resultado fue la fundación, en 2007, de la empresa ahora conocida como MongoDB (antes 10gen).

MongoDB toma su nombre de la palabra en inglés "humongous" (que significa enorme o monstruoso), combinada con la abreviatura de base de datos (DB). Este nombre fue elegido para reflejar su capacidad principal: almacenar y procesar volúmenes gigantescos de datos no estructurados de manera rápida y escalable. Aunque MongoDB se desarrolló para rastrear impresiones en miles de sitios web publicitarios, pronto ganó gran popularidad como almacén de datos flexible en el desarrollo web de código abierto.

4.2. Modelo de Datos

MongoDB utiliza un modelo de datos orientado a documentos. Los datos se organizan en Bases de Datos, que contienen Colecciones, las cuales almacenan Documentos. Estos documentos tienen un esquema dinámico y se representan utilizando el formato JSON/BSON.

Un documento equivale a una fila en SQL mientras que una colección, equivale a una tabla.

MongoDB proporciona validación de esquema, lo que permite crear reglas de validación para los campos, como los tipos de datos permitidos y los rangos de valores.

4.3. Formato JSON frente a BSON

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos simple de leer y escribir para humanos, y simple de interpretar y generar para máquinas. Es un formato de texto independiente del lenguaje, basado en pares clave-valor y listas ordenadas.

Ejemplo simple de una estructura JSON:

```
{  "titulo": "Base de datos NoSQL",
  "autor": "Juan",
  "año": 2026,
  "etiquetas": ["NoSQL", "MongoDB", "JSON"] }
```

MongoDB almacena datos en formato BSON (JSON Binario) tanto internamente como a través de la red. Cualquier cosa que pueda representar en JSON se puede almacenar de forma nativa en MongoDB y recuperarse con la misma facilidad en JSON. BSON añade algunos tipos de datos adicionales (no nativos de JSON), como fechas y datos binarios.

	JSON	BSON
Formato	Texto legible por humanos	Binario(no legible directamente)
Velocidad de parseo	Más lento	Más rápido
Tipos de datos extra	No	Sí (fechas, binario, etc)
Uso en MongoDB	Interfaz de usuario	Almacenamiento interno y red

4.4. Operaciones básicas

Operación	MongoDB	Descripción
CREATE	insertOne() e insertMany()	insertar nuevos documentos en una colección
READ	find() y findOne()	filtrar consultas para especificar qué documentos se desea leer
UPDATE	updateOne(), updateMany() y replaceOne()	actualizar documentos seleccionados por filtro (operación atómica y a nivel documento)
DELETE	deleteOne() y deleteMany()	eliminar documentos seleccionados (operación atómica)

4.5. Ventajas

Las principales ventajas de MongoDB incluyen:

- Facilidad de uso para desarrolladores. Los desarrolladores suelen elegir MongoDB porque es fácil de descargar o acceder en la nube, lo que significa que pueden empezar rápidamente, en parte porque es más fácil trabajar con documentos que crear un modelo de datos y trabajar con tablas.
- Eficiencia. JSON ofrece varias ventajas, como archivos de documentos pequeños y contenido legible para humanos. MongoDB codifica los documentos en formato binario (BSON), que es más compacto y rápido de analizar en comparación con el texto plano.
- Esquemas flexibles. El modelo de datos de documentos de MongoDB permite esquemas flexibles y autodescriptivos, lo que permite que los campos varíen de un documento a otro.
- Lenguaje de consulta sencillo. El lenguaje de consulta de MongoDB está diseñado para que los desarrolladores lo utilicen fácilmente, ofreciendo la posibilidad de realizar consultas complejas e índices para acelerar las consultas de uso común.

4.6. Desventajas

Si bien MongoDB ofrece muchas ventajas, también presenta muchos inconvenientes potenciales:

- Menor rendimiento para transacciones: El soporte para transacciones no es tan robusto como el de las bases de datos relacionales. Para transacciones complejas, sobre todo las que abarcan varias operaciones, pueden tener un rendimiento más bajo y resultar difíciles de implementar.
- Falta de operaciones de unión: MongoDB no admite uniones como en las bases de datos SQL. De igual manera, ofrece opciones que realizan una función similar, pero no es tan eficiente y puede generar consultas más complejas y un rendimiento más lento.
- Uso de memoria: MongoDB almacena sus datos e índices en la memoria RAM, por lo que su rendimiento depende de disponer suficiente memoria RAM. Es por esto que puede consumir más memoria y hardware que otras bases de datos.

4.7. Casos de uso

MongoDB es ideal para:

- Catálogos de comercio electrónico
- Aplicaciones web y móviles
- Gestión de contenido
- Videojuegos
- Big data e inteligencia artificial

Considerar otras alternativas SQL si

La prioridad son las transacciones financieras complejas y estrictas
Si se tienen datos altamente estructurados con relaciones definidas y cruzadas.

4.8. Ejemplo práctico

A continuación se presenta un ejemplo completo utilizando una colección libros. Se realizan las cinco operaciones fundamentales sobre el mismo caso, lo que permite ver cómo interactúan en la práctica.

Paso 1: Insertar documentos (CREATE)

Se insertan dos libros con estructuras similares pero no idénticas, aprovechando el esquema flexible de MongoDB:

```
db.libros.insertMany([
  {
    titulo: "El nombre del viento",
    autor: "Patrick Rothfuss",
    anio: 2007,
    genero: "Fantasía",
    disponible: true,
    precio: 4500
  },
  {
    titulo: "Clean Code",
    autor: "Robert C. Martin",
    anio: 2008,
    genero: "Programación",
    disponible: false,
    precio: 6200,
    edicion: 1
  }
])
```

Paso 2: Consultar documentos (READ)

Buscar todos los libros disponibles, mostrando solo título, autor y precio:

```
db.libros.find(
  { disponible: true },
  { titulo: 1, autor: 1, precio: 1, _id: 0 }
)
```

Resultado esperado:

```
{ titulo: "El nombre del viento", autor: "Patrick Rothfuss",
  precio: 4500 }
```

Paso 3: Actualizar un documento (UPDATE)

Marcar "Clean Code" como disponible y actualizar su precio:

```
db.libros.updateOne(
  { titulo: "Clean Code" },
  {
    $set: { disponible: true },
    $inc: { precio: 300 }
  })
```

Paso 4: Agregar un campo nuevo sin migración (esquema flexible)

Se inserta un nuevo documento con el campo 'resumen'. Los documentos existentes sin ese campo no se ven afectados:

```
db.libros.insertOne({
  titulo: "Dune",
  autor: "Frank Herbert",
  anio: 1965,
  genero: "Ciencia ficción",
  disponible: true,
  precio: 5100,
  resumen: "Una épica de política y ecología en un planeta
desértico"
})
```

Para consultar solo los documentos que incluyen el campo 'resumen':

```
db.libros.find(
  { resumen: { $exists: true } },
  { titulo: 1, resumen: 1, _id: 0 })
```

Paso 5: Eliminar documentos (DELETE)

Eliminar libros que no estén disponibles y cuyo precio sea menor a 5000:

```
db.libros.deleteMany({
  disponible: false,
  precio: { $lt: 5000 }
})
```

5. Comparación con bases de datos SQL

En esta sección se profundiza la comparación entre SQL y NoSQL desde la práctica de modelado, consulta, escalabilidad y performance, tomando como referencia las bases mencionadas en el apartado 2.1, utilizando específicamente base de datos de documentos, el ejemplo de documento JSON del apartado 4.3 y los criterios de uso mencionados en los apartados 3.3, 4.6 y 4.7

5.1. Criterios en las Querys

El documento de ejemplo utilizado para ilustrar JSON y BSON contiene título, año, autor y etiquetas en una única estructura. En un entorno SQL, esa misma información suele distribuirse en tablas relacionadas para evitar redundancia: los datos del autor no se repiten en cada artículo, sino que se referencian mediante un identificador. Así, si el nombre del autor cambia, la actualización se realiza en un solo registro de la tabla de autores.

Para recuperar el título del artículo junto con el nombre del autor, SQL requiere una consulta con unión (JOIN) entre ambas tablas. En NoSQL, utilizando MongoDB, el mismo contenido puede persistir en un solo documento. La lectura se resuelve con una operación directa sobre la colección, sin combinar tablas.

Esta diferencia de enfoque condiciona el diseño del sistema, empezando desde la etapa de planificación de una base de datos, condicionando decisiones como flexibilidad contra velocidad, escalabilidad contra consistencia, y un sinnúmero de variables más.

Ejemplo de la sintaxis de SQL

```
CREATE TABLE autores (
  id INT PRIMARY KEY,
  nombre VARCHAR (100)
);
CREATE TABLE articulos (
  id INT PRIMARY KEY,
  titulo VARCHAR(200),
  anio INT,
  autor_id INT,
  FOREIGN KEY (autor_id) REFERENCES autores(id)
);
```

Se realiza la consulta con union (JOIN) entre ambas tablas:

```
SELECT a.titulo, a.anio, au.nombre
FROM articulos a
JOIN autores au ON a.autor_id = au.id
WHERE a.titulo = 'Base de datos NoSQL'
```

En MongoDB, la lectura se realiza con una operación directa:

Ejemplo de código

```
db.articulos.findOne({ titulo: 'Base de datos NoSQL' });
```

El modelo relacional favorece la coherencia y la ausencia de duplicación, cuando muchas entidades comparten los mismos datos (por ejemplo, un autor con decenas de artículos). El modelo documental prioriza la simplificada lectura de toda la información de una entidad en una sola operación, aunque ello puede generar redundancia si el mismo autor aparece embebido en múltiples documentos, un aspecto ya señalado anteriormente.

5.2. Evolución del esquema

En SQL, incorporar un campo obligatorio para todos los registros implica modificar la tabla y completar los datos existentes. Supongamos que la tabla artículos del apartado 5.1 se le agrega “resumen” como campo obligatorio:

Ejemplo de código

```
SELECT id, titulo, anio FROM articulos WHERE id = 1;
ALTER TABLE articulos ADD resumen TEXT NOT NULL DEFAULT 'Sin resumen';
SELECT id, titulo, resumen FROM articulos WHERE id = 1;
```

El ALTER TABLE impacta toda la tabla: aplicaciones, reportes, consultas deben contemplar el nuevo campo de inmediato.

Ejemplo de código

```
db.articulos.insertOne({
  titulo: "Introducción a Redis",
  autor: "María",
  anio: 2026,
  resumen: "Claves y valores en NoSQL",
  etiquetas: ["NoSQL", "Redis"]
});
```

Y la consulta se simplifica a contener las condiciones que se deben incluir al hacer la lectura:

Ejemplo de código

```
db.articulos.find(
  { resumen: { $exists: true } }, { título: 1, resumen: 1, _id: 0 }
);
```

5.2.1. Escalabilidad

En SQL, ante un aumento de carga suele ampliarse el servidor, resultando en más CPU, RAM o Almacenamiento, aunque el impacto suele variar según la optimización de la base de datos y el servidor. En MongoDB, el modelo documental y distribuido, permite repartir colecciones entre nodos de un cluster, llamado sharding.

Esquema conceptual del sharding:

```
[ App ] → [ Mongos ] → [ Shard A: artículos 1-5000 ]
                        → [ Shard B: artículos 5001-10000 ]
```

5.2.2. Consistencia

En un despliegue con réplicas, MongoDB puede leer desde el nodo primario o desde secundarios. Leer un secundario justo después de una escritura puede devolver datos aún no replicados (consistencia eventual, apartado 3.3.3).

Ejemplo de código de escritura en el primario:

```
db.articulos.insertOne(  
  { titulo: "Nuevo artículo", autor: "Ana", anio: 2026 } );
```

Ejemplo de código de escritura con preferencia en el secundario:

```
db.articulos.find(  
  { titulo: "Nuevo artículo" } ).readPref("secondary");
```

En SQL, en un motor con réplicas síncronas, la lectura en réplica suele garantizar que el dato ya está confirmado (según configuración ACID del motor).

5.2.3. Transacciones

En SQL, utilizando como ejemplo una transferencia entre cuentas, siguiendo las propiedades ACID para la query:

Ejemplo de código en SQL:

```
START TRANSACTION;  
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;  
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;  
-- Si algo falla:  
-- ROLLBACK;  
COMMIT;
```

Si falla la segunda sentencia, ROLLBACK deshace ambas operaciones, por otro lado, MongoDB ofrece las opciones de hacer transacciones multi-documento disponible en despliegues con réplica, o actualizaciones atómicas en un documento. Transacción multi-documento:

Ejemplo de transacción multidocumento:

```
const session = db.getMongo().startSession();
session.startTransaction();

try {
  const cuentas = session.getDatabase("banco").cuentas;
  cuentas.updateOne({ id: 1 }, { $inc: { saldo: -100 } });
  cuentas.updateOne({ id: 2 }, { $inc: { saldo: 100 } });
  session.commitTransaction();
} catch (e) {
  session.abortTransaction();
} finally {
  session.endSession();
}
```

Ejemplo de transacción atómica:

```
db.productos.updateOne(
  { sku: "A-100" },
  { $inc: { stock: -1 } }
);
```

5.3. Comparación de rendimiento entre NoSQL y SQL

El rendimiento depende del modelo de datos, los índices, el volumen de datos, el patrón de acceso, no del motor en sí. Afirmar que NoSQL es siempre más rápido que SQL o que DQL escala siempre peor, sería una forma incorrecta de simplificarlo.

En consultas que recuperan una entidad completa en un solo acceso, MongoDB suele ser eficiente porque toda la información está en un mismo documento. En cambio, cuando la consulta involucra relaciones entre muchas entidades o agregaciones complejas, SQL puede resultar más adecuado gracias a sus capacidades de JOIN nativas. La elección debe evaluarse según las necesidades concretas de cada sistema.

6. Conclusión

A partir de la investigación realizada, se pudo comprender que las bases de datos NoSQL surgieron por la necesidad de manejar grandes cantidades de información y datos con estructuras más variables a las que suelen utilizar bases de datos relacionales tradicionales. A diferencia de SQL, que trabaja principalmente con tablas y relaciones definidas, NoSQL ofrece modelos más flexibles y adaptables a distintos tipos de aplicaciones.

Dentro de las bases de datos NoSQL, las bases de datos documentales se destacan por almacenar la información en documentos JSON o BSON, permitiendo trabajar con esquemas dinámicos y facilitando el desarrollo de aplicaciones modernas. En este contexto, MongoDB es uno de los ejemplos más utilizados debido a su flexibilidad, escalabilidad y facilidad para manejar grandes volúmenes de datos.

A lo largo del trabajo también se pudieron observar algunas diferencias importantes entre SQL y NoSQL, especialmente en la forma de almacenar la información, realizar consultas y escalar los sistemas. Mientras que SQL resulta más adecuado para sistemas con relaciones complejas y alta consistencia de datos, NoSQL ofrece ventajas en aplicaciones que requieren rapidez, flexibilidad y distribución de la información.

En conclusión, ambos modelos de datos tienen ventajas y desventajas, por lo que la elección de uno o el otro dependerá de la necesidad de cada proyecto y del tipo de datos que se necesiten almacenar y procesar.

7. Tabla final de repaso

Concepto	Qué es?	Para qué sirve?	Ejemplo
NoSQL	Familia de bases de datos que no usa el modelo relacional como base principal	Almacenar datos no estructurados o semiestructurados con alta flexibilidad y escala	MongoDB, Redis, Cassandra, Neo4j
MongoDB	Base de datos NoSQL orientada a documentos de código abierto	Guardar y consultar documentos JSON/BSON de forma flexible y distribuida	Una app de e-commerce guarda productos con distintos atributos en una misma colección
Documento	Unidad básica de datos en MongoDB, similar a un objetos JSON	Representar una entidad con todos sus datos en un solo registro	<pre> nombre: "Ana", edad: 30, ciudad: "MDP" </pre>
Colección	Agrupación de documentos relacionados (equivale a una tabla en SQL)	Organizar documentos del mismo tipo o propósito	Colección usuarios con todos los documentos de usuarios del sistema
JSON	Formato texto legible para intercambio de datos usando pared clave-valor	Representar datos de forma estructurada e independiente del lenguaje	<pre> { "titulo": "NoSQL", "año": 2026 } </pre>
BSON	Versión binaria de JSON usada internamente por MongoDB	Almacenar y transmitir datos más rápido y con tipos adicionales (fechas, binario)	MongoDB guarda un campo fecha: ISODate("2026-01-01") en BSON
Sharding	Técnica de distribución de datos entre múltiples servidores (nodos)	Escalar horizontalmente cuando el volumen de datos supera la capacidad de un servidor	Los artículos 1-5000 van al Shard A, los 5001-10000 al Shard B
Consistencia eventual	Modelo donde las réplicas pueden estar desactualizadas por un período breve tras una escritura	Priorizar disponibilidad y rendimiento sobre consistencia inmediata	Leer de un secundario en MongoDB puede devolver datos aún no replicados
Transacciones	Conjunto de operaciones que se ejecutan como una unidad atómica	Garantizar que todas las operaciones se completen o ninguna tenga efecto	Transferencia bancaria: restar de cuenta A y sumar a cuenta B, ambas o ninguna
Escalabilidad horizontal	Capacidad de distribuir la carga agregando más servidores	Manejar más tráfico o datos sin reemplazar el hardware existente	Agregar un nuevo nodo al cluster de MongoDB ante picos de demanda

8. Bibliografía

Google Cloud. (s.f.). ¿Qué es una base de datos documental?
<https://cloud.google.com/discover/what-is-a-document-database?hl=es>

Google Cloud. (s.f.). ¿Qué es MongoDB?
<https://cloud.google.com/discover/what-is-mongodb?hl=es>

IBM. (s.f.). Base de datos. IBM Think.
<https://www.ibm.com/es-es/think/topics/database>

IBM. (s.f.). MongoDB. IBM Think.
<https://www.ibm.com/es-es/think/topics/mongodb>

IBM. (s.f.). Bases de datos NoSQL. IBM Think.
<https://www.ibm.com/es-es/think/topics/nosql-databases>

Integrate.io. (s.f.). The SQL vs NoSQL difference.
<https://www.integrate.io/blog/the-sql-vs-nosql-difference/>

MongoDB. (s.f.). Our story.
<https://www.mongodb.com/company/our-story>

MongoDB. (s.f.). Schema validation. MongoDB Documentation.
<https://www.mongodb.com/es/docs/manual/core/schema-validation/>

Oracle Corporation. (s.f.). What is MongoDB?
<https://www.oracle.com/tr/database/mongodb/>